Chapter 9. GPIO

CAUTION

Under certain conditions, pull-down does not function as expected. For more information, see RP2350-E9.

9.1. Overview

RP2350 has up to 54 multi-functional General Purpose Input / Output (GPIO) pins, divided into two banks:

Bank 0

30 user GPIOs in the QFN-60 package (RP2350A), or 48 user GPIOs in the QFN-80 package

Bank 1

six QSPI IOs, and the USB DP/DM pins

You can control each GPIO from software running on the processors, or by a number of other functional blocks. To meet USB rise and fall specifications, the analogue characteristics of the USB pins differ from the GPIO pads. As a result, we do not include them in the 54 GPIO total. However, you can still use them for UART, I2C, or processor-controlled GPIO through the single-cycle IO subsystem (SIO).

In a typical use case, the QSPI IOs are used to execute code from an external flash device, leaving 30 or 48 Bank 0 GPIOs for the programmer to use. The QSPI pins might become available for general purpose use when booting the chip from internal OTP, or controlling the chip externally through SWD in an IO expander application.

All GPIOs support digital input and output. Several Bank 0 GPIOs can also be used as inputs to the chip's Analogue to Digital Converter (ADC):

- GPIOs 26 through 29 inclusive (four total) in the QFN-60 package
- GPIOs 40 through 47 (eight total) in the QFN-80 package

Bank 0 supports the following functions:

- Software control via SIO — Section 3.1.3, "GPIO control"
- Programmable IO (PIO) — Chapter 11, *PIO*
- 2 × SPI — Section 12.3, "SPI"
- 2 × UART — Section 12.1, "UART"
- 2 × I2C (two-wire serial interface) — Section 12.2, "I2C"
- 8 × two-channel PWM in the QFN-60 package, or 12 × in QFN-80 — Section 12.5, "PWM"
- 2 × external clock inputs — Section 8.1.2.4, "External clocks"
- 4 × general purpose clock output — Section 8.1, "Overview"
- 4 × input to ADC in the QFN-60 package, or 8 × in QFN-80 — Section 12.4, "ADC and Temperature Sensor"
- 1 × HSTX high-speed interface — Section 12.11, "HSTX"
- 1 × auxiliary QSPI chip select, for a second XIP device — Section 12.14, "QSPI memory interface (QMI)"
- CoreSight execution trace output — Section 3.5.7, "Trace"
- USB VBUS management — Section 12.7.3.10, "VBUS control"
- External interrupt requests, level or edge-sensitive — Section 9.5, "Interrupts"

Bank 1 contains the QSPI and USB DP/DM pins and supports the following functions:

- Software control via SIO — Section 3.1.3, “GPIO control”
- Flash execute in place (Section 4.4, “External flash and PSRAM (XIP)”) via QSPI Memory Interface (QMI) — Section 12.14, “QSPI memory interface (QMI)”
- UART — Section 12.1, “UART”
- I2C (two-wire serial interface) — Section 12.2, “I2C”

The logical structure of an example IO is shown in Figure 41.

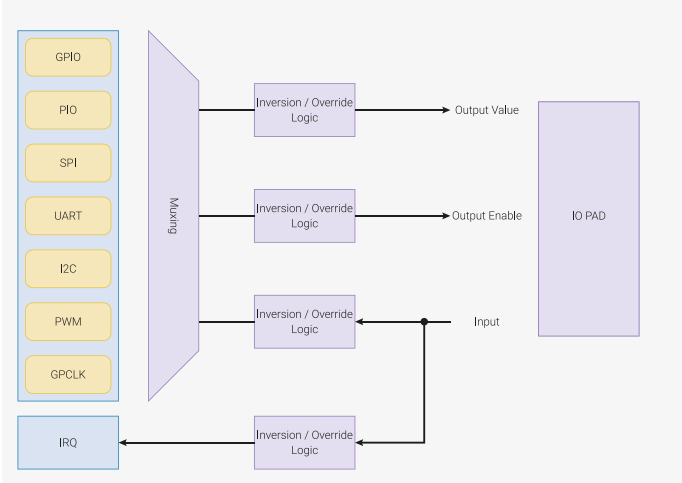


Figure 41. Logical structure of a GPIO. Each GPIO can be controlled by one of a number of peripherals, or by software control registers in the SIO. The function select (FSEL) selects which peripheral output is in control of the GPIO’s direction and output level, and which peripheral input can see this GPIO’s input level. These three signals (output level, output enable, input level) can also be inverted or forced high or low, using the GPIO control registers.

9.2. Changes from RP2040

RP2350 GPIO differs from RP2040 in the following ways:

- 18 more GPIOs in the QFN-80 package
- Addition of a third PIO to GPIO functions
- USB DP/DM pins can be used as GPIO
- Addition of isolation register to pad registers (preserves pad state while in a low power state, cleared by software on power up)
- Changed default reset state of pad controls
- Both Secure and Non-secure access to GPIOs (see Section 10.6)
- Double the number of GPIO interrupts to differentiate between Secure and Non-secure
- Interrupt summary registers added so you can quickly see which GPIOs have pending interrupts

9.3. Reset state

At first power up, Bank 0 IOs (GPIOs 0 through 29 in the QFN-60 package, and GPIOs 0 through 47 in the QFN-80 package) assume the following state:

- Output buffer is high-impedance
- Input buffer is disabled
- Pulled low
- Isolation latches are set to latched (Section 9.7)

The pad output disable bit (GPIO0.OD) for each pad is clear at reset, but the IO muxing is reset to the null function,

which ensures that the output buffer is high-impedance.

IMPORTANT

> The pad reset state is different from RP2040, which only disables digital inputs on GPIOs 26 through 29 (as of version B2) and does not have isolation latches. Applications must enable the pad input (GPIO0.IE = 1) and disable pad isolation latches (GPIO0.ISO = 0) before using the pads for digital I/O. The `gpio_set_function()` SDK function performs these tasks automatically.

Bank 1 IOs have the same reset state as Bank 0 GPIOs, except for the input enable (IE) resetting to 1, and different pull-up/pull-down states: SCK, SD0 and SD1 are pull-down, but SD2, SD3 and CSn are pull-up.

NOTE

> To use a Bank 0 GPIO as a second chip select, you need an external pull-up to ensure the second QSPI device does not power up with its chip select asserted.

The pads return to the reset state on any of the following:

- A brownout reset
- Asserting the RUN pin low
- Setting SW-DP CDBGRSTREQ via SWD
- Setting RP-AP rescue reset via SWD

If a pad's isolation latches are in the latched state (Section 9.7) then resetting the PADS and IO registers does not physically return the pad to its reset state. The isolation latches prevent upstream signals from propagating to the pad. Clear the ISO bit to allow signals to propagate.

9.4. Function select

To allocate a function to a GPIO, write to the `FUNCSEL` field in the `CTRL` register corresponding to the pin. For a list of GPIOs and corresponding registers, see Table 645. For an example, see GPIO0_CTRL. The descriptions for the functions listed in this table can be found in Table 646.

Each GPIO can only select one function at a time. Each peripheral input (e.g. UART0 RX) should only be selected by one GPIO at a time. If you connect the same peripheral input to multiple GPIOs, the peripheral sees the logical OR of these GPIO inputs.

Table 645. General Purpose Input/Output (GPIO) Bank 0 Functions

GPIO	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11
0		SPI0 RX	UART0 TX	I2C0 SDA	PWM0 A	SIO	PIO0	PIO1	PIO2	QMI CS1n	USB OVCUR DET	
1		SPI0 CSn	UART0 RX	I2C0 SCL	PWM0 B	SIO	PIO0	PIO1	PIO2	TRACECLK	USB VBUS DET	
2		SPI0 SCK	UART0 CTS	I2C1 SDA	PWM1 A	SIO	PIO0	PIO1	PIO2	TRACEDATA0	USB VBUS EN	UART0 TX
3		SPI0 TX	UART0 RTS	I2C1 SCL	PWM1 B	SIO	PIO0	PIO1	PIO2	TRACEDATA1	USB OVCUR DET	UART0 RX
4		SPI0 RX	UART1 TX	I2C0 SDA	PWM2 A	SIO	PIO0	PIO1	PIO2	TRACEDATA2	USB VBUS DET	
5		SPI0 CSn	UART1 RX	I2C0 SCL	PWM2 B	SIO	PIO0	PIO1	PIO2	TRACEDATA3	USB VBUS EN	
6		SPI0 SCK	UART1 CTS	I2C1 SDA	PWM3 A	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	UART1 TX
7		SPI0 TX	UART1 RTS	I2C1 SCL	PWM3 B	SIO	PIO0	PIO1	PIO2		USB VBUS DET	UART1 RX
8		SPI1 RX	UART1 TX	I2C0 SDA	PWM4 A	SIO	PIO0	PIO1	PIO2	QMI CS1n	USB VBUS EN	
9		SPI1 CSn	UART1 RX	I2C0 SCL	PWM4 B	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	
10		SPI1 SCK	UART1 CTS	I2C1 SDA	PWM5 A	SIO	PIO0	PIO1	PIO2		USB VBUS DET	UART1 TX
11		SPI1 TX	UART1 RTS	I2C1 SCL	PWM5 B	SIO	PIO0	PIO1	PIO2		USB VBUS EN	UART1 RX
12	HSTX	SPI1 RX	UART0 TX	I2C0 SDA	PWM6 A	SIO	PIO0	PIO1	PIO2	CLOCK GPIN0	USB OVCUR DET	
13	HSTX	SPI1 CSn	UART0 RX	I2C0 SCL	PWM6 B	SIO	PIO0	PIO1	PIO2	CLOCK GPOUT0	USB VBUS DET	
14	HSTX	SPI1 SCK	UART0 CTS	I2C1 SDA	PWM7 A	SIO	PIO0	PIO1	PIO2	CLOCK GPIN1	USB VBUS EN	UART0 TX
15	HSTX	SPI1 TX	UART0 RTS	I2C1 SCL	PWM7 B	SIO	PIO0	PIO1	PIO2	CLOCK GPOUT1	USB OVCUR DET	UART0 RX
16	HSTX	SPI0 RX	UART0 TX	I2C0 SDA	PWM0 A	SIO	PIO0	PIO1	PIO2		USB VBUS DET	
17	HSTX	SPI0 CSn	UART0 RX	I2C0 SCL	PWM0 B	SIO	PIO0	PIO1	PIO2		USB VBUS EN	
18	HSTX	SPI0 SCK	UART0 CTS	I2C1 SDA	PWM1 A	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	UART0 TX
19	HSTX	SPI0 TX	UART0 RTS	I2C1 SCL	PWM1 B	SIO	PIO0	PIO1	PIO2	QMI CS1n	USB VBUS DET	UART0 RX
20		SPI0 RX	UART1 TX	I2C0 SDA	PWM2 A	SIO	PIO0	PIO1	PIO2	CLOCK GPIN0	USB VBUS EN	
21		SPI0 CSn	UART1 RX	I2C0 SCL	PWM2 B	SIO	PIO0	PIO1	PIO2	CLOCK GPOUT0	USB OVCUR DET	
22		SPI0 SCK	UART1 CTS	I2C1 SDA	PWM3 A	SIO	PIO0	PIO1	PIO2	CLOCK GPIN1	USB VBUS DET	UART1 TX

GPIO	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11
23		SPI0 TX	UART1 RTS	I2C1 SCL	PWM3 B	SIO	PIO0	PIO1	PIO2	CLOCK GPOUT1	USB VBUS EN	UART1 RX
24		SPI1 RX	UART1 TX	I2C0 SDA	PWM4 A	SIO	PIO0	PIO1	PIO2	CLOCK GPOUT2	USB OVCUR DET	
25		SPI1 CSn	UART1 RX	I2C0 SCL	PWM4 B	SIO	PIO0	PIO1	PIO2	CLOCK GPOUT3	USB VBUS DET	
26		SPI1 SCK	UART1 CTS	I2C1 SDA	PWM5 A	SIO	PIO0	PIO1	PIO2		USB VBUS EN	UART1 TX
27		SPI1 TX	UART1 RTS	I2C1 SCL	PWM5 B	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	UART1 RX
28		SPI1 RX	UART0 TX	I2C0 SDA	PWM6 A	SIO	PIO0	PIO1	PIO2		USB VBUS DET	
29		SPI1 CSn	UART0 RX	I2C0 SCL	PWM6 B	SIO	PIO0	PIO1	PIO2		USB VBUS EN	
GPIOs 30 through 47 are QFN-80 only:												
30		SPI1 SCK	UART0 CTS	I2C1 SDA	PWM7 A	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	UART0 TX
31		SPI1 TX	UART0 RTS	I2C1 SCL	PWM7 B	SIO	PIO0	PIO1	PIO2		USB VBUS DET	UART0 RX
32		SPI0 RX	UART0 TX	I2C0 SDA	PWM8 A	SIO	PIO0	PIO1	PIO2		USB VBUS EN	
33		SPI0 CSn	UART0 RX	I2C0 SCL	PWM8 B	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	
34		SPI0 SCK	UART0 CTS	I2C1 SDA	PWM9 A	SIO	PIO0	PIO1	PIO2		USB VBUS DET	UART0 TX
35		SPI0 TX	UART0 RTS	I2C1 SCL	PWM9 B	SIO	PIO0	PIO1	PIO2		USB VBUS EN	UART0 RX
36		SPI0 RX	UART1 TX	I2C0 SDA	PWM10 A	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	
37		SPI0 CSn	UART1 RX	I2C0 SCL	PWM10 B	SIO	PIO0	PIO1	PIO2		USB VBUS DET	
38		SPI0 SCK	UART1 CTS	I2C1 SDA	PWM11 A	SIO	PIO0	PIO1	PIO2		USB VBUS EN	UART1 TX
39		SPI0 TX	UART1 RTS	I2C1 SCL	PWM11 B	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	UART1 RX
40		SPI1 RX	UART1 TX	I2C0 SDA	PWM8 A	SIO	PIO0	PIO1	PIO2		USB VBUS DET	
41		SPI1 CSn	UART1 RX	I2C0 SCL	PWM8 B	SIO	PIO0	PIO1	PIO2		USB VBUS EN	
42		SPI1 SCK	UART1 CTS	I2C1 SDA	PWM9 A	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	UART1 TX
43		SPI1 TX	UART1 RTS	I2C1 SCL	PWM9 B	SIO	PIO0	PIO1	PIO2		USB VBUS DET	UART1 RX
44		SPI1 RX	UART0 TX	I2C0 SDA	PWM10 A	SIO	PIO0	PIO1	PIO2		USB VBUS EN	

GPIO	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11
45		SPI1 CSn	UART0 RX	I2C0 SCL	PWM10 B	SIO	PIO0	PIO1	PIO2		USB OVCUR DET	
46		SPI1 SCK	UART0 CTS	I2C1 SDA	PWM11 A	SIO	PIO0	PIO1	PIO2		USB VBUS DET	UART0 TX
47		SPI1 TX	UART0 RTS	I2C1 SCL	PWM11 B	SIO	PIO0	PIO1	PIO2	QMI CS1n	USB VBUS EN	UART0 RX

Table 646. GPIO User Bank function descriptions

Function Name	Description
SPIx	Connect one of the internal PL022 SPI peripherals to GPIO.
UARTx	Connect one of the internal PL011 UART peripherals to GPIO.
I2Cx	Connect one of the internal DW I2C peripherals to GPIO.
PWMx A/B	Connect a PWM slice to GPIO. There are twelve PWM slices, each with two output channels (A/B). The B pin can also be used as an input, for frequency and duty cycle measurement.
SIO	Software control of GPIO from the Single-cycle IO (SIO) block. The SIO function (F5) must be selected for the processors to *drive* a GPIO, but the input is always connected, so software can check the state of GPIOs at any time.
PIOx	Connect one of the programmable IO blocks (PIO) to GPIO. PIO can implement a wide variety of interfaces, and has its own internal pin mapping hardware, allowing flexible placement of digital interfaces on Bank 0 GPIOs. The PIO function (F6, F7, F8) must be selected for PIO to *drive* a GPIO, but the input is always connected, so the PIOs can always see the state of all pins.
HSTX	Connect the high-speed transmit peripheral (HSTX) to GPIO.
CLOCK GPINx	General purpose clock inputs. Can be routed to a number of internal clock domains on RP2350, e.g. to provide a 1Hz clock for the AON Timer, or can be connected to an internal frequency counter.
CLOCK GPOUTx	General purpose clock outputs. Can drive a number of internal clocks (including PLL outputs) onto GPIOs, with optional integer divide.
TRACECLK, TRACEDATAx	CoreSight execution trace output from Cortex-M33 processors (Arm-only).
USB OVCUR DET/VBUS DET/VBUS EN	USB power control signals to/from the internal USB controller.
QMI CS1n	Auxiliary chip select for QSPI bus, to allow execute-in-place from an additional flash or PSRAM device.

Bank 1 function select operates identically to Bank 0, but its registers are in a different register block, starting with USBPHY_DP_CTRL.

Table 647. GPIO Bank 1 Functions

Pin	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11
USB DP			UART1 TX	I2C0 SDA		SIO						
USB DM			UART1 RX	I2C0 SCL		SIO						
QSPI SCK	QMI SCK		UART1 CTS	I2C1 SDA		SIO						UART1 TX
QSPI CSn	QMI CS0n		UART1 RTS	I2C1 SCL		SIO						UART1 RX
QSPI SD0	QMI SD0		UART0 TX	I2C0 SDA		SIO						
QSPI SD1	QMI SD1		UART0 RX	I2C0 SCL		SIO						
QSPI SD2	QMI SD2		UART0 CTS	I2C1 SDA		SIO						UART0 TX
QSPI SD3	QMI SD3		UART0 RTS	I2C1 SCL		SIO						UART0 RX

Table 648. GPIO bank 1 function descriptions

Function Name	Description
UARTx	Connect one of the internal PL011 UART peripherals to GPIO.
I2Cx	Connect one of the internal DW I2C peripherals to GPIO.

Function Name	Description
SIO	Software control of GPIO, from the single-cycle IO (SIO) block. The SIO function (F5) must be selected for the processors to *drive* a GPIO, but the input is always connected, so software can check the state of GPIOs at any time.
QMI	QSPI memory interface peripheral, used for execute-in-place from external QSPI flash or PSRAM memory devices.

The six QSPI Bank GPIO pins are typically used by the XIP peripheral to communicate with an external flash device. However, there are two scenarios where the pins can be used as software-controlled GPIOs:

- If a SPI or Dual-SPI flash device is used for execute-in-place, then the SD2 and SD3 pins are not used for flash access, and can be used for other GPIO functions on the circuit board.
- If RP2350 is used in a flashless configuration (USB and OTP boot only), then all six pins can be used for software-controlled GPIO functions.

9.5. Interrupts

An interrupt can be generated for every GPIO pin in four scenarios:

- Level High: the GPIO pin is a logical 1
- Level Low: the GPIO pin is a logical 0
- Edge High: the GPIO has transitioned from a logical 0 to a logical 1
- Edge Low: the GPIO has transitioned from a logical 1 to a logical 0

The level interrupts are not latched. This means that if the pin is a logical 1 and the level high interrupt is active, it will become inactive as soon as the pin changes to a logical 0. The edge interrupts are stored in the INTR register and can be cleared by writing to the INTR register.

There are enable, status, and force registers for three interrupt destinations: proc 0, proc 1, and dormant_wake. For proc 0 the registers are enable (PROC0_INTE0), status (PROC0_INTS0), and force (PROC0_INTF0). Dormant wake is used to wake the ROSC or XOSC up from dormant mode. See Section 6.5.6.2 for more information on dormant mode.

There is an interrupt output for each combination of IO bank, IRQ destination, and security domain. In total there are twelve such outputs:

- IO Bank 0 to dormant wake (Secure and Non-secure)
- IO Bank 0 to proc 0 (Secure and Non-secure)
- IO Bank 0 to proc 1 (Secure and Non-secure)
- IO QSPI to dormant wake (Secure and Non-secure)
- IO QSPI to proc 0 (Secure and Non-secure)
- IO QSPI to proc 1 (Secure and Non-secure)

Each interrupt output has its own array of enable registers (INTE) that configures which GPIO events cause the interrupt to assert. The interrupt asserts when at least one enabled event occurs, and de-asserts when all enabled events have been acknowledged via the relevant INTR register.

This means the user can watch for several GPIO events at once.

Summary registers can be used to quickly check for pending GPIO interrupts. See IRQSUMMARY_PROC0_NONSECURE0 for an example.

9.6. Pads

CAUTION

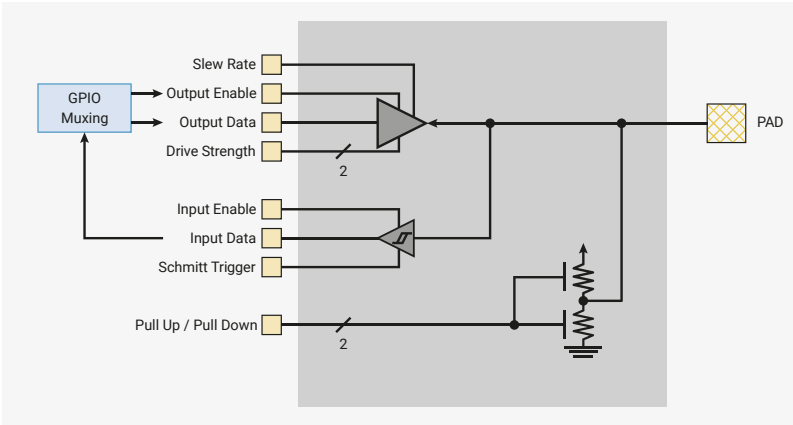
Under certain conditions, pull-down does not function as expected. For more information, see RP2350-E9.

Each GPIO is connected off-chip via a **pad**. Pads are the electrical interface between the chip's internal logic and external circuitry. They translate signal voltage levels, support higher currents and offer some protection against electrostatic discharge (ESD) events. You can adjust pad electrical behaviour to meet the requirements of external circuitry in the following ways:

- Output drive strength can be set to 2mA, 4mA, 8mA or 12mA.
- Output slew rate can be set to slow or fast.
- Input hysteresis (Schmitt trigger mode) can be enabled.
- A pull-up or pull-down can be enabled, to set the output signal level when the output driver is disabled.
- The input buffer can be disabled, to reduce current consumption when the pad is unused, unconnected or connected to an analogue signal.

An example pad is shown in Figure 42.

Figure 42. Diagram of a single IO pad.



The pad's Output Enable, Output Data and Input Data ports connect, via the IO mux, to the function controlling the pad. All other ports are controlled from the pad control register. You can use this register to disable the pad's output driver by overriding the Output Enable signal from the function controlling the pad. See GPIO0 for an example of a pad control register.

Both the output signal level and acceptable input signal level at the pad are determined by the digital IO supply (IOVDD). IOVDD can be any nominal voltage between 1.8V and 3.3V, but to meet specification when powered at 1.8V, the pad input thresholds must be adjusted by writing a 1 to the pad `VOLTAGE_SELECT` registers. By default, the pad input thresholds are valid for an IOVDD voltage between 2.5V and 3.3V. Using a voltage of 1.8V with the default input thresholds is a safe operating mode, but it will result in input thresholds that don't meet specification.

WARNING

Using IOVDD voltages greater than 1.8V, with the input thresholds set for 1.8V may result in damage to the chip.

Pad input threshold are adjusted on a per bank basis, with separate `VOLTAGE_SELECT` registers for the pads associated with the User IO bank (IO Bank 0) and the QSPI IO bank. However, both banks share the same digital IO supply (IOVDD), so both register should always be set to the same value.

Pad register details are available in Section 9.11.3, "Pad Control - User Bank" and Section 9.11.4, "Pad Control - QSPI Bank".

9.6.1. Bus keeper mode

For each pad, only the pull-up or the pull-down resistor can be enabled at any given time. It is impossible to enable both simultaneously. Instead, if you set both the GPIO0.PDE and GPIO0.PUE bits simultaneously then you enable **bus keeper** mode, where the pad is:

- Pulled up when its input is high.
- Pulled down when its input is low.

When the output buffer is disabled, and the pad is not driven by any external source, this mode weakly retains the pad's current logical state. The pad does not float to mid-rail.

Bus keeper mode relies on control logic in the switched core domain, so does not function when the core is powered down. Rather, powering down the core when bus keeper mode is enabled latches the current output controls (pull-up or pull-down) in the pad isolation latches, as described in Section 9.7.

9.7. Pad isolation latches

RP2350 features extended low-power states that allow all internal logic, with the exception of POWMAN and some CoreSight debug logic, to fully power down under software control. This includes powering down all peripherals, the IO muxing, and the pad control registers, which brings with it the risk that pad signals may experience unwanted transitions when entering and exiting low-power states.

To ensure that pad states are well-defined at all times, all signals passing from the switched core power domain to the pads pass through **isolation latches**. In normal operation, the latches are transparent, so the pads are controlled fully by logic inside the switched core power domain, such as UARTs or the processors. However, when the ISO bit for each pad is set (e.g. GPIO0.ISO) or the switched core domain is powered down, the control signals currently presented to that pad are **latched** until the isolation is disabled. This includes the output enable state, output high/low level, and pull-up/pull-down resistor enable. The input signal from the pad back into the switched core domain is not isolated.

Consequently, when switched core logic is powered down, all Bank 0 and Bank 1 pads maintain the output state they held immediately before the power down, unless overridden by always-on logic in POWMAN. When the switched core power domain powers back up, all the GPIO ISO bits reset to 1, so the pre-power down state continues to be maintained until user software starts up and clears the ISO bit to indicate it is ready to use the pad again. Pads whose IO muxing has not yet been set up can be left isolated indefinitely, and will maintain their pre-power down state.

when software has finished setting up the IO muxing for a given pad, and the peripheral that is to be muxed in, the ISO bit should be cleared. At this point the isolation latches will become transparent again: output signals passing through the IO muxing block are now reflected in the pad output state, so peripherals can communicate with the outside world. This process allows the switched core domain to be power cycled without causing any transitions on the pad outputs that may interfere with the operation of external hardware connected to the pads.

NOTE

> Non-SDK applications ported from RP2040 must clear the ISO bit before using a GPIO, as this feature was not present on RP2040. The SDK automatically clears the ISO bit when `gpio_set_function()` is called.

The isolation latches themselves are reset by the always-on power domain reset, namely any one of:

- Power-on reset
- Brownout reset
- RUN pin being asserted low
- SW-DP CDBGRSTREQ
- RP-AP rescue reset

The latches reset to the reset value of the signal being isolated. For example, on Bank 0 GPIOs, the input enable control

(GPIO0.IE) resets to 0 (input-disabled), so the isolation latches for these signals also take a reset value of 0. Resetting the isolation latch forces the pad to assume its reset state *even if it is currently isolated*.

The ISO control bits (e.g. GPIO0.ISO) are reset by the top-level switched core domain isolation signal, which is asserted by POWMAN before powering down the switched core domain and de-asserted after it is powered up. This means that entering and exiting a sleep state where the switched core domain is unpowered leaves all GPIOs isolated after power up; you can then re-engage them individually. The ISO control bits are not reset by the PADS register block reset driven by the RESETS control registers: resetting the PADS register block returns non-isolated pads to their reset state, but has no effect on isolated pads.

9.8. Processor GPIO controls (SIO)

The single-cycle IO subsystem (Section 3.1) contains memory-mapped GPIO registers. The processors can use these to perform input/output operations on GPIOs:

- The GPIO_OUT and GPIO_HI_OUT registers set the output level: 1 = high, 0 = low
- The GPIO_OE and GPIO_HI_OE registers set the output enable: 1 = output, 0 = input
- The GPIO_IN and GPIO_HI_IN registers read the GPIO inputs

These registers are all 32 bits in size. The low registers (e.g. GPIO_OUT) connect to GPIOs 0 through 31, and the high registers (e.g. GPIO_HI_OUT) connect to GPIOs 32 through 47, the QSPI pads, and the USB DM/DP pads.

For the output and output enable registers to take effect, the SIO function must be selected on each GPIO (function `5`). However, the GPIO input registers read back the GPIO input values even when the SIO function is not selected, so the processor can always check the input state of any pin.

The SIO GPIO registers are shared between the two processors and between the Secure and Non-secure security domains. This avoids programming errors introduced by selecting multiple GPIO functions for access from different contexts.

Non-secure code's view of the SIO registers is restricted by the Non-secure GPIO mask defined in GPIO_NSMASK0 and GPIO_NSMASK1. Non-secure writes to Secure GPIOs are ignored. Non-secure reads of Secure GPIOs return 0.

These registers are documented in more detail in the SIO GPIO register section (Section 3.1.3).

The DMA cannot access registers in the SIO subsystem. The recommended method to DMA to GPIOs is a PIO program that continuously transfers TX FIFO data to the GPIO outputs, which provides more consistent timing than DMA directly into GPIO registers.

9.9. GPIO coprocessor port

Coprocessor port `0` on each Cortex-M33 processor connects to a GPIO coprocessor interface. These coprocessor instructions provide fast access to the SIO GPIO registers from Arm software:

- The equivalent of any SIO GPIO register access is a single instruction, without having to materialise a 32-bit register address beforehand
- An indexed write operation on any single GPIO is a single instruction
- 64 bits can be read/written in a single instruction

This reduces the timing impact of GPIO accesses on surrounding software, for example when GPIO tracing has been added to interrupt handlers diagnose complex timing issues.

Both Secure and Non-secure code may access the coprocessor. Non-secure code sees a restricted view of the GPIO registers, defined by ACCESSCTRL GPIO_NSMASK0/1.

The GPIO coprocessor instruction set is documented in Section 3.6.1.

9.10. Software examples

9.10.1. Select an IO function

An IO pin can perform many different functions and must be configured before use. For example, you may want it to be a `UART_TX` pin, or a `PWM` output. The SDK provides `gpio_set_function` for this purpose. Many SDK examples call `gpio_set_function` early on to enable printing to a UART.

The SDK starts by defining a structure to represent the registers of IO Bank 0, the User IO bank. Each IO has a status register, followed by a control register. For `N` IOs, the SDK instantiates the structure containing a status and control register as `io[N]` to repeat it `N` times.

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2350/hardware_structs/include/hardware/structs/io_bank0.h Lines 179 - 445

```
179 typedef struct {
180     io_bank0_status_ctrl_hw_t io[48];
181
182     uint32_t _pad0[32];
183
184     // (Description copied from array index 0 register IO_BANK0_IRQSUMMARY_PROC0_SECURE0
    applies similarly to other array indexes)
185     _REG_(IO_BANK0_IRQSUMMARY_PROC0_SECURE0_OFFSET) // IO_BANK0_IRQSUMMARY_PROC0_SECURE0
186     // 0x80000000 [31]    GPIO31       (0)
187     // 0x40000000 [30]    GPIO30       (0)
188     // 0x20000000 [29]    GPIO29       (0)
189     // 0x10000000 [28]    GPIO28       (0)
190     // 0x08000000 [27]    GPIO27       (0)
191     // 0x04000000 [26]    GPIO26       (0)
192     // 0x02000000 [25]    GPIO25       (0)
193     // 0x01000000 [24]    GPIO24       (0)
194     // 0x00800000 [23]    GPIO23       (0)
195     // 0x00400000 [22]    GPIO22       (0)
196     // 0x00200000 [21]    GPIO21       (0)
197     // 0x00100000 [20]    GPIO20       (0)
198     // 0x00080000 [19]    GPIO19       (0)
199     // 0x00040000 [18]    GPIO18       (0)
200     // 0x00020000 [17]    GPIO17       (0)
201     // 0x00010000 [16]    GPIO16       (0)
202     // 0x00008000 [15]    GPIO15       (0)
203     // 0x00004000 [14]    GPIO14       (0)
204     // 0x00002000 [13]    GPIO13       (0)
205     // 0x00001000 [12]    GPIO12       (0)
206     // 0x00000800 [11]    GPIO11       (0)
207     // 0x00000400 [10]    GPIO10       (0)
208     // 0x00000200 [9]     GPIO9        (0)
209     // 0x00000100 [8]     GPIO8        (0)
210     // 0x00000080 [7]     GPIO7        (0)
211     // 0x00000040 [6]     GPIO6        (0)
212     // 0x00000020 [5]     GPIO5        (0)
213     // 0x00000010 [4]     GPIO4        (0)
214     // 0x00000008 [3]     GPIO3        (0)
215     // 0x00000004 [2]     GPIO2        (0)
216     // 0x00000002 [1]     GPIO1        (0)
217     // 0x00000001 [0]     GPIO0        (0)
218     io_ro_32 irqsummary_proc0_secure[2];
219
220     // (Description copied from array index 0 register IO_BANK0_IRQSUMMARY_PROC0_NONSECURE0
    applies similarly to other array indexes)
221     _REG_(IO_BANK0_IRQSUMMARY_PROC0_NONSECURE0_OFFSET) //
    IO_BANK0_IRQSUMMARY_PROC0_NONSECURE0
222     // 0x80000000 [31]    GPIO31       (0)
```

```
    // 0x40000000 [30]    GPIO30       (0)
    // 0x20000000 [29]    GPIO29       (0)
    // 0x10000000 [28]    GPIO28       (0)
    // 0x08000000 [27]    GPIO27       (0)
    // 0x04000000 [26]    GPIO26       (0)
    // 0x02000000 [25]    GPIO25       (0)
    // 0x01000000 [24]    GPIO24       (0)
    // 0x00800000 [23]    GPIO23       (0)
    // 0x00400000 [22]    GPIO22       (0)
    // 0x00200000 [21]    GPIO21       (0)
    // 0x00100000 [20]    GPIO20       (0)
    // 0x00080000 [19]    GPIO19       (0)
    // 0x00040000 [18]    GPIO18       (0)
    // 0x00020000 [17]    GPIO17       (0)
    // 0x00010000 [16]    GPIO16       (0)
    // 0x00008000 [15]    GPIO15       (0)
    // 0x00004000 [14]    GPIO14       (0)
    // 0x00002000 [13]    GPIO13       (0)
    // 0x00001000 [12]    GPIO12       (0)
    // 0x00000800 [11]    GPIO11       (0)
    // 0x00000400 [10]    GPIO10       (0)
    // 0x00000200 [9]     GPIO9        (0)
    // 0x00000100 [8]     GPIO8        (0)
    // 0x00000080 [7]     GPIO7        (0)
    // 0x00000040 [6]     GPIO6        (0)
    // 0x00000020 [5]     GPIO5        (0)
    // 0x00000010 [4]     GPIO4        (0)
    // 0x00000008 [3]     GPIO3        (0)
    // 0x00000004 [2]     GPIO2        (0)
    // 0x00000002 [1]     GPIO1        (0)
    // 0x00000001 [0]     GPIO0        (0)
    io_ro_32 irqsummary_proc0_nonsecure[2];

    // (Description copied from array index 0 register IO_BANK0_IRQSUMMARY_PROC1_SECURE0 applies similarly to other array indexes)
    _REG_(IO_BANK0_IRQSUMMARY_PROC1_SECURE0_OFFSET) // IO_BANK0_IRQSUMMARY_PROC1_SECURE0
    // 0x80000000 [31]    GPIO31       (0)
    // 0x40000000 [30]    GPIO30       (0)
    // 0x20000000 [29]    GPIO29       (0)
    // 0x10000000 [28]    GPIO28       (0)
    // 0x08000000 [27]    GPIO27       (0)
    // 0x04000000 [26]    GPIO26       (0)
    // 0x02000000 [25]    GPIO25       (0)
    // 0x01000000 [24]    GPIO24       (0)
    // 0x00800000 [23]    GPIO23       (0)
    // 0x00400000 [22]    GPIO22       (0)
    // 0x00200000 [21]    GPIO21       (0)
    // 0x00100000 [20]    GPIO20       (0)
    // 0x00080000 [19]    GPIO19       (0)
    // 0x00040000 [18]    GPIO18       (0)
    // 0x00020000 [17]    GPIO17       (0)
    // 0x00010000 [16]    GPIO16       (0)
    // 0x00008000 [15]    GPIO15       (0)
    // 0x00004000 [14]    GPIO14       (0)
    // 0x00002000 [13]    GPIO13       (0)
    // 0x00001000 [12]    GPIO12       (0)
    // 0x00000800 [11]    GPIO11       (0)
    // 0x00000400 [10]    GPIO10       (0)
    // 0x00000200 [9]     GPIO9        (0)
    // 0x00000100 [8]     GPIO8        (0)
    // 0x00000080 [7]     GPIO7        (0)
    // 0x00000040 [6]     GPIO6        (0)
    // 0x00000020 [5]     GPIO5        (0)
    // 0x00000010 [4]     GPIO4        (0)
```

```
    // 0x00000008 [3]     GPIO3        (0)
    // 0x00000004 [2]     GPIO2        (0)
    // 0x00000002 [1]     GPIO1        (0)
    // 0x00000001 [0]     GPIO0        (0)
    io_ro_32 irqsummary_proc1_secure[2];

    // (Description copied from array index 0 register IO_BANK0_IRQSUMMARY_PROC1_NONSECURE0 applies similarly to other array indexes)
    _REG_(IO_BANK0_IRQSUMMARY_PROC1_NONSECURE0_OFFSET) // IO_BANK0_IRQSUMMARY_PROC1_NONSECURE0
    // 0x80000000 [31]    GPIO31       (0)
    // 0x40000000 [30]    GPIO30       (0)
    // 0x20000000 [29]    GPIO29       (0)
    // 0x10000000 [28]    GPIO28       (0)
    // 0x08000000 [27]    GPIO27       (0)
    // 0x04000000 [26]    GPIO26       (0)
    // 0x02000000 [25]    GPIO25       (0)
    // 0x01000000 [24]    GPIO24       (0)
    // 0x00800000 [23]    GPIO23       (0)
    // 0x00400000 [22]    GPIO22       (0)
    // 0x00200000 [21]    GPIO21       (0)
    // 0x00100000 [20]    GPIO20       (0)
    // 0x00080000 [19]    GPIO19       (0)
    // 0x00040000 [18]    GPIO18       (0)
    // 0x00020000 [17]    GPIO17       (0)
    // 0x00010000 [16]    GPIO16       (0)
    // 0x00008000 [15]    GPIO15       (0)
    // 0x00004000 [14]    GPIO14       (0)
    // 0x00002000 [13]    GPIO13       (0)
    // 0x00001000 [12]    GPIO12       (0)
    // 0x00000800 [11]    GPIO11       (0)
    // 0x00000400 [10]    GPIO10       (0)
    // 0x00000200 [9]     GPIO9        (0)
    // 0x00000100 [8]     GPIO8        (0)
    // 0x00000080 [7]     GPIO7        (0)
    // 0x00000040 [6]     GPIO6        (0)
    // 0x00000020 [5]     GPIO5        (0)
    // 0x00000010 [4]     GPIO4        (0)
    // 0x00000008 [3]     GPIO3        (0)
    // 0x00000004 [2]     GPIO2        (0)
    // 0x00000002 [1]     GPIO1        (0)
    // 0x00000001 [0]     GPIO0        (0)
    io_ro_32 irqsummary_proc1_nonsecure[2];

    // (Description copied from array index 0 register IO_BANK0_IRQSUMMARY_DORMANT_WAKE_SECURE0 applies similarly to other array indexes)
    _REG_(IO_BANK0_IRQSUMMARY_DORMANT_WAKE_SECURE0_OFFSET) // IO_BANK0_IRQSUMMARY_DORMANT_WAKE_SECURE0
    // 0x80000000 [31]    GPIO31       (0)
    // 0x40000000 [30]    GPIO30       (0)
    // 0x20000000 [29]    GPIO29       (0)
    // 0x10000000 [28]    GPIO28       (0)
    // 0x08000000 [27]    GPIO27       (0)
    // 0x04000000 [26]    GPIO26       (0)
    // 0x02000000 [25]    GPIO25       (0)
    // 0x01000000 [24]    GPIO24       (0)
    // 0x00800000 [23]    GPIO23       (0)
    // 0x00400000 [22]    GPIO22       (0)
    // 0x00200000 [21]    GPIO21       (0)
    // 0x00100000 [20]    GPIO20       (0)
    // 0x00080000 [19]    GPIO19       (0)
    // 0x00040000 [18]    GPIO18       (0)
    // 0x00020000 [17]    GPIO17       (0)
    // 0x00010000 [16]    GPIO16       (0)
```

```
    // 0x00008000 [15]    GPIO15       (0)
    // 0x00004000 [14]    GPIO14       (0)
    // 0x00002000 [13]    GPIO13       (0)
    // 0x00001000 [12]    GPIO12       (0)
    // 0x00000800 [11]    GPIO11       (0)
    // 0x00000400 [10]    GPIO10       (0)
    // 0x00000200 [9]     GPIO9        (0)
    // 0x00000100 [8]     GPIO8        (0)
    // 0x00000080 [7]     GPIO7        (0)
    // 0x00000040 [6]     GPIO6        (0)
    // 0x00000020 [5]     GPIO5        (0)
    // 0x00000010 [4]     GPIO4        (0)
    // 0x00000008 [3]     GPIO3        (0)
    // 0x00000004 [2]     GPIO2        (0)
    // 0x00000002 [1]     GPIO1        (0)
    // 0x00000001 [0]     GPIO0        (0)
    io_ro_32 irqsummary_dormant_wake_secure[2];

    // (Description copied from array index 0 register IO_BANK0_IRQSUMMARY_DORMANT_WAKE_NONSECURE0 applies similarly to other array indexes)
    _REG_(IO_BANK0_IRQSUMMARY_DORMANT_WAKE_NONSECURE0_OFFSET) // IO_BANK0_IRQSUMMARY_DORMANT_WAKE_NONSECURE0
    // 0x80000000 [31]    GPIO31       (0)
    // 0x40000000 [30]    GPIO30       (0)
    // 0x20000000 [29]    GPIO29       (0)
    // 0x10000000 [28]    GPIO28       (0)
    // 0x08000000 [27]    GPIO27       (0)
    // 0x04000000 [26]    GPIO26       (0)
    // 0x02000000 [25]    GPIO25       (0)
    // 0x01000000 [24]    GPIO24       (0)
    // 0x00800000 [23]    GPIO23       (0)
    // 0x00400000 [22]    GPIO22       (0)
    // 0x00200000 [21]    GPIO21       (0)
    // 0x00100000 [20]    GPIO20       (0)
    // 0x00080000 [19]    GPIO19       (0)
    // 0x00040000 [18]    GPIO18       (0)
    // 0x00020000 [17]    GPIO17       (0)
    // 0x00010000 [16]    GPIO16       (0)
    // 0x00008000 [15]    GPIO15       (0)
    // 0x00004000 [14]    GPIO14       (0)
    // 0x00002000 [13]    GPIO13       (0)
    // 0x00001000 [12]    GPIO12       (0)
    // 0x00000800 [11]    GPIO11       (0)
    // 0x00000400 [10]    GPIO10       (0)
    // 0x00000200 [9]     GPIO9        (0)
    // 0x00000100 [8]     GPIO8        (0)
    // 0x00000080 [7]     GPIO7        (0)
    // 0x00000040 [6]     GPIO6        (0)
    // 0x00000020 [5]     GPIO5        (0)
    // 0x00000010 [4]     GPIO4        (0)
    // 0x00000008 [3]     GPIO3        (0)
    // 0x00000004 [2]     GPIO2        (0)
    // 0x00000002 [1]     GPIO1        (0)
    // 0x00000001 [0]     GPIO0        (0)
    io_ro_32 irqsummary_dormant_wake_nonsecure[2];

    // (Description copied from array index 0 register IO_BANK0_INTR0 applies similarly to other array indexes)
    _REG_(IO_BANK0_INTR0_OFFSET) // IO_BANK0_INTR0
    // Raw Interrupts
    // 0x80000000 [31]    GPIO7_EDGE_HIGH (0)
    // 0x40000000 [30]    GPIO7_EDGE_LOW (0)
    // 0x20000000 [29]    GPIO7_LEVEL_HIGH (0)
    // 0x10000000 [28]    GPIO7_LEVEL_LOW (0)
```

```
    // 0x08000000 [27]    GPIO6_EDGE_HIGH (0)
    // 0x04000000 [26]    GPIO6_EDGE_LOW (0)
    // 0x02000000 [25]    GPIO6_LEVEL_HIGH (0)
    // 0x01000000 [24]    GPIO6_LEVEL_LOW (0)
    // 0x00800000 [23]    GPIO5_EDGE_HIGH (0)
    // 0x00400000 [22]    GPIO5_EDGE_LOW (0)
    // 0x00200000 [21]    GPIO5_LEVEL_HIGH (0)
    // 0x00100000 [20]    GPIO5_LEVEL_LOW (0)
    // 0x00080000 [19]    GPIO4_EDGE_HIGH (0)
    // 0x00040000 [18]    GPIO4_EDGE_LOW (0)
    // 0x00020000 [17]    GPIO4_LEVEL_HIGH (0)
    // 0x00010000 [16]    GPIO4_LEVEL_LOW (0)
    // 0x00008000 [15]    GPIO3_EDGE_HIGH (0)
    // 0x00004000 [14]    GPIO3_EDGE_LOW (0)
    // 0x00002000 [13]    GPIO3_LEVEL_HIGH (0)
    // 0x00001000 [12]    GPIO3_LEVEL_LOW (0)
    // 0x00000800 [11]    GPIO2_EDGE_HIGH (0)
    // 0x00000400 [10]    GPIO2_EDGE_LOW (0)
    // 0x00000200 [9]     GPIO2_LEVEL_HIGH (0)
    // 0x00000100 [8]     GPIO2_LEVEL_LOW (0)
    // 0x00000080 [7]     GPIO1_EDGE_HIGH (0)
    // 0x00000040 [6]     GPIO1_EDGE_LOW (0)
    // 0x00000020 [5]     GPIO1_LEVEL_HIGH (0)
    // 0x00000010 [4]     GPIO1_LEVEL_LOW (0)
    // 0x00000008 [3]     GPIO0_EDGE_HIGH (0)
    // 0x00000004 [2]     GPIO0_EDGE_LOW (0)
    // 0x00000002 [1]     GPIO0_LEVEL_HIGH (0)
    // 0x00000001 [0]     GPIO0_LEVEL_LOW (0)
    io_rw_32 intr[6];

    union {
        struct {
            io_bank0_irq_ctrl_hw_t proc0_irq_ctrl;
            io_bank0_irq_ctrl_hw_t proc1_irq_ctrl;
            io_bank0_irq_ctrl_hw_t dormant_wake_irq_ctrl;
        };
        io_bank0_irq_ctrl_hw_t irq_ctrl[3];
    };
} io_bank0_hw_t;
```

A similar structure is defined for the pad control registers for IO bank 1. By default, all pads come out of reset ready to use, with input enabled and output disable set to 0. Regardless, `gpio_set_function` in the SDK sets the input enable and clears the output disable to engage the pad's IO buffers and connect internal signals to the outside world. Finally, the desired function select is written to the IO control register (see GPIO0_CTRL for an example of an IO control register).

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_gpio/gpio.c Lines 36 - 53

```
36 // Select function for this GPIO, and ensure input/output are enabled at the pad.
37 // This also clears the input/output/irq override bits.
38 void gpio_set_function(uint gpio, gpio_function_t fn) {
39     check_gpio_param(gpio);
40     invalid_params_if(HARDWARE_GPIO, ((uint32_t)fn << IO_BANK0_GPIO0_CTRL_FUNCSEL_LSB) &
   ~IO_BANK0_GPIO0_CTRL_FUNCSEL_BITS);
41     // Set input enable on, output disable off
42     hw_write_masked(&pads_bank0_hw->io[gpio],
43                     PADS_BANK0_GPIO0_IE_BITS,
44                     PADS_BANK0_GPIO0_IE_BITS | PADS_BANK0_GPIO0_OD_BITS
45     );
46     // Zero all fields apart from fsel; we want this IO to do what the peripheral tells it.
47     // This doesn't affect e.g. pullup/pulldown, as these are in pad controls.
48     io_bank0_hw->io[gpio].ctrl = fn << IO_BANK0_GPIO0_CTRL_FUNCSEL_LSB;
49     // Remove pad isolation now that the correct peripheral is in control of the pad
```

```
    hw_clear_bits(&pads_bank0_hw->io[gpio], PADS_BANK0_GPIO0_ISO_BITS);
}
```

9.10.2. Enable a GPIO interrupt

The SDK provides a method of being interrupted when a GPIO pin changes state:

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_gpio/gpio.c Lines 186 - 196

```
186 void gpio_set_irq_enabled(uint gpio, uint32_t events, bool enabled) {
187     // either this call disables the interrupt or callback should already be set.
188     // this protects against enabling the interrupt without callback set
189     assert(!enabled || irq_has_handler(IO_IRQ_BANK0));
190
191     // Separate mask/force/status per-core, so check which core called, and
192     // set the relevant IRQ controls.
193     io_bank0_irq_ctrl_hw_t *irq_ctrl_base = get_core_num() ?
194                                         &io_bank0_hw->proc1_irq_ctrl : &io_bank0_hw-
    >proc0_irq_ctrl;
195     _gpio_set_irq_enabled(gpio, events, enabled, irq_ctrl_base);
196 }
```

`gpio_set_irq_enabled` uses a lower level function `_gpio_set_irq_enabled`:

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_gpio/gpio.c Lines 173 - 184

```
173 static void _gpio_set_irq_enabled(uint gpio, uint32_t events, bool enabled,
    io_bank0_irq_ctrl_hw_t *irq_ctrl_base) {
174     // Clear stale events which might cause immediate spurious handler entry
175     gpio_acknowledge_irq(gpio, events);
176
177     io_rw_32 *en_reg = &irq_ctrl_base->inte[gpio / 8];
178     events <<= 4 * (gpio % 8);
179
180     if (enabled)
181         hw_set_bits(en_reg, events);
182     else
183         hw_clear_bits(en_reg, events);
184 }
```

The user provides a pointer to a callback function that is called when the GPIO event happens. An example application that uses this system is `hello_gpio_irq`:

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/gpio/hello_gpio_irq/hello_gpio_irq.c

```
 1 /**
 2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
 3  *
 4  * SPDX-License-Identifier: BSD-3-Clause
 5  */
 6
 7 #include <stdio.h>
 8 #include "pico/stdlib.h"
 9 #include "hardware/gpio.h"
10
11 #define GPIO_WATCH_PIN 2
12
```